Dart Sass and Hugo Themes

Rohit Goswami · 2023-06-17 · webdev

> *Quick notes on variables, `dart-sass` and `hugo`*

Background

Recently I decided that it was annoying for me as a user to not know which links on my site were internal and which led off to an external site.

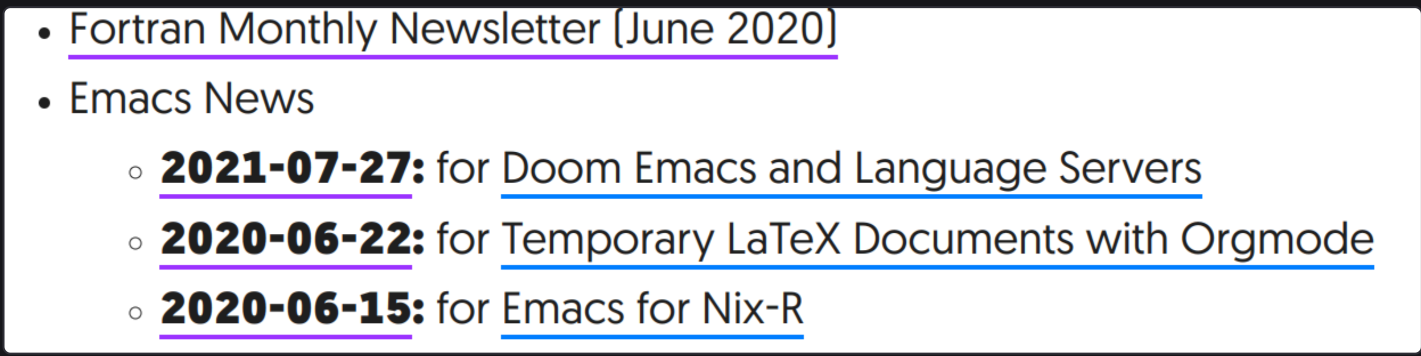

Figure 1: The desired end-result, blue for internal links, purple for external links, colors chosen with [ColorHexa](https://www.colorhexa.com/)

However, since `hugo` requires `baseurl` for generating meaningful `rss` and `sitemap` files, this meant either hardcoding the variable into my theme, or figuring out how to pass variables from my `config.toml` to my theme's (generated) `css`.

This about the latter, and is heavily based off of this post on the Hugo Discourse.

The CSS

The crux of the implementation is simply the use of attribute selectors:

```
a {
  &[href^="/"],
  &[href*="rgoswami.me"],
  &[href^="#"] {
    border-bottom: #0380ff 0.125em solid;
  }
  &:not([href^="/"]):not([href*="rgoswami.me"]):not([href^="#"]) {
    border-bottom: #9b36ff 0.125em solid;
  }
}
```

Where the main sticking point is being able to pass the name of the site from the configuration. Although this could be done with `libsass` and `hugo` as detailed in the discourse post, it is conceptually clunky, since partials are not respected when using `ExecuteAsTemplate` with hugo-pipes. A better SCSS processor could probably take in a better configuration option object.... Which leads to the next section.

Dart Sass

At some point which I didn't have time to figure out, `libsass` got phased out in favor of Dart Sass and includes (among other features) a nifty `math` module. This had a few side effects:

- I could swap my hand-kanged `exp` function for `math.exp()` and also had to transition to using `calc()` more (as in 3938ac2).
- I could pass variables from `hugo` to `scss` and from there to `css` for attribute selectors (as in 804d417)

CI Updates

For various reasons, `hugo` (even the extended version) does not seem to include `dart-sass-embedded` though the `snap` package does (as noted here on Discourse).

```
- name: Setup Hugo
  uses: peaceiris/actions-hugo@v2
  with:
    hugo-version: '0.111.3'
    extended: true
- name: Install Dart Sass Embedded
  run: sudo snap install dart-sass-embedded
```

Local Usage

`hugo serve` will correctly compile and load the theme if `hugo env` recognizes the `dart-sass-embedded` installation:

```
$ hugo env
hugo v0.112.3+extended linux/amd64 BuildDate=unknown
GOOS="linux"
GOARCH="amd64"
GOVERSION="go1.20.4"
github.com/sass/libsass="3.6.5"
github.com/webmproject/libwebp="v1.2.4"
github.com/sass/dart-sass-embedded/protocol="1.2.0"
github.com/sass/dart-sass-embedded/compiler="1.54.8"
github.com/sass/dart-sass-embedded/implementation="1.54.8"
```

`dart-sass-embedded` is a node package, which means that `npm` can be used:

```
DART_SASS_VERSION=1.54.8 npm install sass-embedded-linux-x64@${DART_SASS_VERSION}
PATH=$PATH:./node_modules/sass-embedded-linux-x64/dart-sass-embedded/ hugo env
```

Personally I ended up using `micromamba` to grab `dart-sass` as well.

Passing Variables

User Perspective

The theme users simply updates their configuration to include the relevant variables:

```
baseurl = "https://rgoswami.me/"
[params.style]
site_url = "rgoswami.me" # Same as baseurl
```

Theme Dev Perspective

With the `hugo` asset processing pipeline for `dart-sass`, the relevant options need to be (minimally) set:

```
{{ $options := (dict "targetPath" "main.css" "transpiler" "dartsass" "vars" site.Params.style) }}
{{ $style := resources.Get "scss/main.scss" |  resources.ToCSS $options | resources.Minify |
resources.Fingerprint }}
<link rel="stylesheet" href="{{ $style.RelPermalink }}">
```

Where `vars` takes a section corresponding to the user-defined section (`style` in this case). Usage within `scss` partials looks like:

```
@use "hugo:vars" as h;
$site-url: h.$site_url;
$link-color: #0380ff;
$external-link-color: #9b36ff;
```

Which can finally be used to elegantly setup attribute selector based colors:

```
a {
  &[href^="/"],
  &[href*="#{$site-url}"],
  &[href^="#"] {
    border-bottom: $link-color 0.125em solid;
  }
  &:not([href^="/"]):not([href*="#{$site-url}"]):not([href^="#"]) {
    border-bottom: $external-link-color 0.125em solid;
  }
}
```

Conclusions

Of the many maintenance burdens I have accrued over the years, one of the least often discussed is the underlying theme used for the site. Some day that will be rectified. I think web design via vanilla JS, S(CSS) and HTML is a pretty nifty gateway into other languages (interpreted and compiled). The approach could be (and perhaps should be) extended to color other links and sites differently (e.g. all my content on other sites in blue). For now this works well enough at any rate.